Project Checkpoint 4

Goal:

- Use the best model selected from Checkpoint 3
- Perform multiple independent production runs
- Perform N-fold cross-validation
- Report mean and standard deviation of model performance
- Finalize the production-ready configuration

In [1]:
```
import warnings
warnings.filterwarnings("ignore", category=UserWarning)

import copy
import numpy as np
import random
import matplotlib.pyplot as plt
from tqdm import tqdm

import torch
import torch.nn as nn
import torchani
```

Device

In [2]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("Device:", device)
```

```
Device: cuda
```

In [3]:
```
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

Dataset Loading

In [4]:
```
def load_ani_dataset(dspath):
    species_order = ['H', 'C', 'N', 'O']
    energy_shifter = torchani.utils.EnergyShifter(None)

    dataset = torchani.data.load(dspath)
    dataset = dataset.subtract_self_energies(energy_shifter, species_order)
    dataset = dataset.species_to_indices(species_order)
```

```
    dataset = dataset.shuffle()
    return dataset
```

AEV Computer

In [5]:
```
def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5

    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)

    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
        0.19634954, 0.58904862, 0.98174770, 1.37444680,
        1.76714590, 2.15984490, 2.55254400, 2.94524300
    ], dtype=torch.float, device=device)

    return torchani.AEVComputer(
        Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, 4
    )

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print("AEV dimension:", aev_dim)
```

```
AEV dimension: 384
```

Best Configuration

In [6]:
```
BEST_CONFIG = {
    "hidden_layers": [128, 64],
    "activation": "relu",
    "batch_size": 4096,
    "learning_rate": 1e-3,
    "epoch": 10,
    "l2": 1e-5
}

print("Best configuration from Checkpoint 3:")
for k, v in BEST_CONFIG.items():
    print(f"{k}: {v}")
```

```
Best configuration from Checkpoint 3:
hidden_layers: [128, 64]
activation: relu
batch_size: 4096
learning_rate: 0.001
epoch: 10
l2: 1e-05
```

CV Configuration

In [7]:
```python
CV_CONFIG = {
    "hidden_layers": [128, 64],
    "activation": "relu",
    "batch_size": 2048,
    "learning_rate": 1e-3,
    "epoch": 20,
    "l2": 1e-5
}

print("\nCV configuration:")
for k, v in CV_CONFIG.items():
    print(f"{k}: {v}")
```

```
CV configuration:
hidden_layers: [128, 64]
activation: relu
batch_size: 2048
learning_rate: 0.001
epoch: 20
l2: 1e-05
```

Flexible atomic network

In [8]:
```python
class AtomicNet(nn.Module):
    def __init__(self, input_dim, hidden_layers, activation="relu"):
        super().__init__()

        layers = []
        prev_dim = input_dim

        if activation == "relu":
            act_layer = nn.ReLU
        elif activation == "tanh":
            act_layer = nn.Tanh
        else:
            raise ValueError("Unsupported activation. Use 'relu' or 'tanh'.")

        for hidden_dim in hidden_layers:
            layers.append(nn.Linear(prev_dim, hidden_dim))
            layers.append(act_layer())
            prev_dim = hidden_dim

        layers.append(nn.Linear(prev_dim, 1))
        self.layers = nn.Sequential(*layers)
```

```
    def forward(self, x):
        return self.layers(x)
```

Hyperparameters

In [9]:
```
def build_model(hidden_layers, activation="relu"):
    net_H = AtomicNet(aev_dim, hidden_layers, activation=activation)
    net_C = AtomicNet(aev_dim, hidden_layers, activation=activation)
    net_N = AtomicNet(aev_dim, hidden_layers, activation=activation)
    net_O = AtomicNet(aev_dim, hidden_layers, activation=activation)

    ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])

    model = nn.Sequential(
        aev_computer,
        ani_net
    ).to(device)

    return model
```

Trainer

In [10]:
```
class ANITrainer:
    def __init__(self, model, batch_size, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.batch_size = batch_size
        self.optimizer = torch.optim.Adam(
            self.model.parameters(),
            lr=learning_rate,
            weight_decay=l2
        )
        self.epoch = epoch

    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        print("Initialize training data...")
        train_data_loader = train_data.collate(self.batch_size).cache()

        loss_func = nn.MSELoss()

        train_loss_list = []
        val_loss_list = []
        lowest_val_loss = np.inf
        best_weights = None

        for i in tqdm(range(self.epoch), leave=True):
            self.model.train()
            train_epoch_loss = 0.0
            train_epoch_count = 0
```

```
            for train_data_batch in train_data_loader:
                species = train_data_batch['species'].to(device)
                coords = train_data_batch['coordinates'].to(device)
                true_energies = train_data_batch['energies'].to(device).float()

                _, pred_energies = self.model((species, coords))
                batch_loss = loss_func(pred_energies, true_energies)

                self.optimizer.zero_grad()
                batch_loss.backward()
                self.optimizer.step()

                batch_importance = species.shape[0]
                train_epoch_loss += batch_loss.item() * batch_importance
                train_epoch_count += batch_importance

            train_epoch_loss /= train_epoch_count
            val_epoch_loss = self.evaluate(val_data, draw_plot=False)

            train_loss_list.append(train_epoch_loss)
            val_loss_list.append(val_epoch_loss)

            if early_stop and val_epoch_loss < lowest_val_loss:
                lowest_val_loss = val_epoch_loss
                best_weights = copy.deepcopy(self.model.state_dict())

        if draw_curve:
            fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
            ax.set_yscale("log")
            ax.plot(train_loss_list, label="Train")
            ax.plot(val_loss_list, label="Validation")
            ax.set_xlabel("Epoch")
            ax.set_ylabel("MSE Loss")
            ax.legend()

        if early_stop and best_weights is not None:
            self.model.load_state_dict(best_weights)

        return train_loss_list, val_loss_list

    def evaluate(self, data, draw_plot=False):
        data_loader = data.collate(self.batch_size).cache()

        loss_func = nn.MSELoss()
        total_loss = 0.0
        total_count = 0

        if draw_plot:
            true_energies_all = []
            pred_energies_all = []

        self.model.eval()

        with torch.no_grad():
            for batch_data in data_loader:
```

```
                species = batch_data['species'].to(device)
                coords = batch_data['coordinates'].to(device)
                true_energies = batch_data['energies'].to(device).float()

                _, pred_energies = self.model((species, coords))
                batch_loss = loss_func(pred_energies, true_energies)

                batch_importance = species.shape[0]
                total_loss += batch_loss.item() * batch_importance
                total_count += batch_importance

                if draw_plot:
                    true_energies_all.append(true_energies.detach().cpu().numpy().f
                    pred_energies_all.append(pred_energies.detach().cpu().numpy().f

        avg_loss = total_loss / total_count

        if draw_plot:
            true_energies_all = np.concatenate(true_energies_all)
            pred_energies_all = np.concatenate(pred_energies_all)

            hartree2kcalmol = 627.5094738898777
            mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2

            fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
            ax.scatter(true_energies_all, pred_energies_all,
                       label=f"MAE: {mae:.2f} kcal/mol", s=2)
            ax.set_xlabel("Ground Truth Energy (Hartree)")
            ax.set_ylabel("Predicted Energy (Hartree)")

            xmin, xmax = ax.get_xlim()
            ymin, ymax = ax.get_ylim()
            vmin, vmax = min(xmin, ymin), max(xmax, ymax)
            ax.set_xlim(vmin, vmax)
            ax.set_ylim(vmin, vmax)
            ax.plot([vmin, vmax], [vmin, vmax], color='red')
            ax.legend()

            print(f"MAE: {mae:.4f} kcal/mol")

        return avg_loss

    def evaluate_mae_kcalmol(self, data):
        data_loader = data.collate(self.batch_size).cache()

        true_energies_all = []
        pred_energies_all = []

        self.model.eval()

        with torch.no_grad():
            for batch_data in data_loader:
                species = batch_data['species'].to(device)
                coords = batch_data['coordinates'].to(device)
                true_energies = batch_data['energies'].to(device).float()
```

```
                _, pred_energies = self.model((species, coords))

                true_energies_all.append(true_energies.detach().cpu().numpy().flatt
                pred_energies_all.append(pred_energies.detach().cpu().numpy().flatt

        true_energies_all = np.concatenate(true_energies_all)
        pred_energies_all = np.concatenate(pred_energies_all)

        hartree2kcalmol = 627.5094738898777
        mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcal
        return mae
```

Load Data

In [11]:
```
dataset_path = "./ani_gdb_s01_to_s04.h5"

set_seed(42)
dataset = load_ani_dataset(dataset_path)
train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)

print("Main train/val/test split ready.")
```

```
Main train/val/test split ready.
```

Part 1: Multiple runs on chosen configuration

In [12]:
```
def run_single_production_trial(train_data, val_data, test_data, seed):
    print(f"\nRunning production trial with seed = {seed}")
    set_seed(seed)

    model = build_model(
        hidden_layers=BEST_CONFIG["hidden_layers"],
        activation=BEST_CONFIG["activation"]
    )

    trainer = ANITrainer(
        model=model,
        batch_size=BEST_CONFIG["batch_size"],
        learning_rate=BEST_CONFIG["learning_rate"],
        epoch=BEST_CONFIG["epoch"],
        l2=BEST_CONFIG["l2"]
    )

    train_losses, val_losses = trainer.train(
        train_data,
        val_data,
        early_stop=True,
        draw_curve=False
    )

    val_mse = trainer.evaluate(val_data, draw_plot=False)
    val_mae = trainer.evaluate_mae_kcalmol(val_data)
    test_mse = trainer.evaluate(test_data, draw_plot=False)
```

```
    test_mae = trainer.evaluate_mae_kcalmol(test_data)

    result = {
        "seed": seed,
        "train_losses": train_losses,
        "val_losses": val_losses,
        "val_mse": val_mse,
        "val_mae": val_mae,
        "test_mse": test_mse,
        "test_mae": test_mae,
        "trainer": trainer
    }

    print(f"Validation MAE: {val_mae:.4f} kcal/mol")
    print(f"Test MAE:       {test_mae:.4f} kcal/mol")
    return result

production_results = []
seeds = [1, 2, 3]

for seed in seeds:
    production_results.append(
        run_single_production_trial(train_data, val_data, test_data, seed)
    )
```

```
Running production trial with seed = 1
Sequential - Number of parameters: 230404
Initialize training data...
100%|██████████| 10/10 [01:36<00:00,  9.61s/it]
Validation MAE: 1.6286 kcal/mol
Test MAE:       1.6223 kcal/mol

Running production trial with seed = 2
Sequential - Number of parameters: 230404
Initialize training data...
100%|██████████| 10/10 [01:00<00:00,  6.04s/it]
Validation MAE: 1.6604 kcal/mol
Test MAE:       1.6492 kcal/mol

Running production trial with seed = 3
Sequential - Number of parameters: 230404
Initialize training data...
100%|██████████| 10/10 [00:57<00:00,  5.75s/it]
Validation MAE: 1.6574 kcal/mol
Test MAE:       1.6453 kcal/mol
```

Multiple Runs Summary

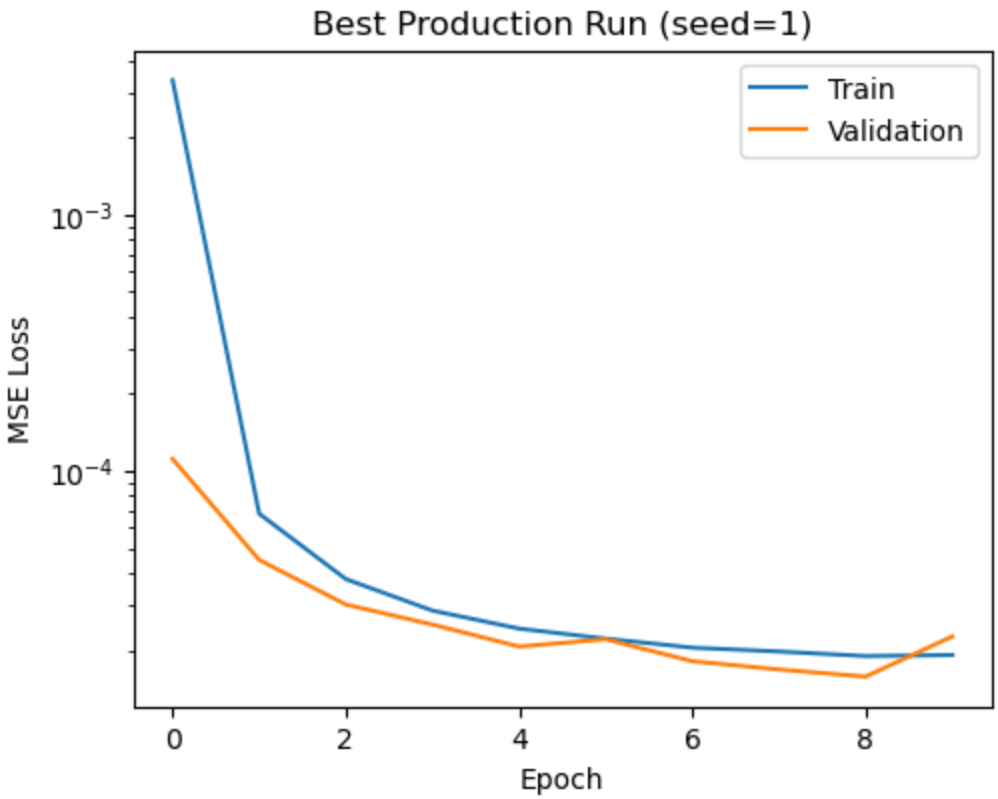
In [13]:
```
print("\nMultiple-run production summary:")
for res in production_results:
    print(
        f"Seed {res['seed']}: "
        f"Val MAE = {res['val_mae']:.4f} kcal/mol, "
        f"Test MAE = {res['test_mae']:.4f} kcal/mol"
```

```
    )

val_maes = [res["val_mae"] for res in production_results]
test_maes = [res["test_mae"] for res in production_results]

print("\nAggregate production performance:")
print(f"Validation MAE mean ± std = {np.mean(val_maes):.4f} ± {np.std(val_maes):.4f
print(f"Test MAE mean ± std       = {np.mean(test_maes):.4f} ± {np.std(test_maes):.
```

```
Multiple-run production summary:
Seed 1: Val MAE = 1.6286 kcal/mol, Test MAE = 1.6223 kcal/mol
Seed 2: Val MAE = 1.6604 kcal/mol, Test MAE = 1.6492 kcal/mol
Seed 3: Val MAE = 1.6574 kcal/mol, Test MAE = 1.6453 kcal/mol

Aggregate production performance:
Validation MAE mean ± std = 1.6488 ± 0.0143 kcal/mol
Test MAE mean ± std       = 1.6389 ± 0.0118 kcal/mol
```

In [14]:
```
best_run = min(production_results, key=lambda x: x["val_mae"])

fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
ax.set_yscale("log")
ax.plot(best_run["train_losses"], label="Train")
ax.plot(best_run["val_losses"], label="Validation")
ax.set_xlabel("Epoch")
ax.set_ylabel("MSE Loss")
ax.set_title(f"Best Production Run (seed={best_run['seed']})")
ax.legend()

print(f"\nBest production run seed: {best_run['seed']}")
print(f"Best production validation MAE: {best_run['val_mae']:.4f} kcal/mol")
print(f"Best production test MAE:       {best_run['test_mae']:.4f} kcal/mol")
```

```
Best production run seed: 1
Best production validation MAE: 1.6286 kcal/mol
Best production test MAE:       1.6223 kcal/mol
```



Part 2: K-fold Cross Validation

In [15]:
```python
print("\nCreating development subset for cross-validation...")

def materialize_examples(data_loader, max_batches=20):
    examples = []

    for i, batch in enumerate(data_loader):
        if i >= max_batches:
            break

        species = batch["species"].cpu()
        coordinates = batch["coordinates"].cpu()
        energies = batch["energies"].cpu()

        batch_size_local = species.shape[0]

        for j in range(batch_size_local):
            examples.append({
                "species": species[j].clone(),
                "coordinates": coordinates[j].clone(),
                "energies": energies[j].clone()
            })

    return examples

# Start from the main training split so we do not contaminate the held-out test set
cv_source_loader = train_data.collate(BEST_CONFIG["batch_size"]).cache()
cv_examples = materialize_examples(cv_source_loader, max_batches=50)
```

```
print(f"Number of materialized examples for CV: {len(cv_examples)}")


# ------------------------------------------------------------
# Shuffled in-memory dataset and loader
# ------------------------------------------------------------
class InMemoryDataset:
    def __init__(self, examples):
        self.examples = examples

    def __len__(self):
        return len(self.examples)

    def collate(self, batch_size, shuffle=False):
        return InMemoryBatchLoader(self.examples, batch_size, shuffle=shuffle)


class InMemoryBatchLoader:
    def __init__(self, examples, batch_size, shuffle=False):
        self.examples = examples
        self.batch_size = batch_size
        self.shuffle = shuffle

    def cache(self):
        return self

    def __iter__(self):
        indices = np.arange(len(self.examples))
        if self.shuffle:
            np.random.shuffle(indices)

        for start in range(0, len(indices), self.batch_size):
            batch_idx = indices[start:start + self.batch_size]
            batch = [self.examples[idx] for idx in batch_idx]

            species = torch.stack([item["species"] for item in batch], dim=0)
            coordinates = torch.stack([item["coordinates"] for item in batch], dim=
            energies = torch.stack([item["energies"] for item in batch], dim=0)

            yield {
                "species": species,
                "coordinates": coordinates,
                "energies": energies
            }


# ------------------------------------------------------------
# Trainer subclass for in-memory shuffled CV training
# ------------------------------------------------------------
class CVTrainer(ANITrainer):
    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        print("Initialize CV training data...")
        loss_func = nn.MSELoss()

        train_loss_list = []
```

```
        val_loss_list = []
        lowest_val_loss = np.inf
        best_weights = None

        for i in tqdm(range(self.epoch), leave=True):
            self.model.train()
            train_epoch_loss = 0.0
            train_epoch_count = 0

            # IMPORTANT: shuffled batches every epoch
            train_data_loader = train_data.collate(self.batch_size, shuffle=True).c

            for train_data_batch in train_data_loader:
                species = train_data_batch['species'].to(device)
                coords = train_data_batch['coordinates'].to(device)
                true_energies = train_data_batch['energies'].to(device).float()

                _, pred_energies = self.model((species, coords))
                batch_loss = loss_func(pred_energies, true_energies)

                self.optimizer.zero_grad()
                batch_loss.backward()
                self.optimizer.step()

                batch_importance = species.shape[0]
                train_epoch_loss += batch_loss.item() * batch_importance
                train_epoch_count += batch_importance

            train_epoch_loss /= train_epoch_count
            val_epoch_loss = self.evaluate(val_data, draw_plot=False)

            train_loss_list.append(train_epoch_loss)
            val_loss_list.append(val_epoch_loss)

            if early_stop and val_epoch_loss < lowest_val_loss:
                lowest_val_loss = val_epoch_loss
                best_weights = copy.deepcopy(self.model.state_dict())

        if draw_curve:
            fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
            ax.set_yscale("log")
            ax.plot(train_loss_list, label="Train")
            ax.plot(val_loss_list, label="Validation")
            ax.set_xlabel("Epoch")
            ax.set_ylabel("MSE Loss")
            ax.legend()

        if early_stop and best_weights is not None:
            self.model.load_state_dict(best_weights)

        return train_loss_list, val_loss_list


# ------------------------------------------------------------
# K-fold splitting
# ------------------------------------------------------------
```

```
def make_k_folds_indices(n_examples, k=3, seed=42):
    rng = np.random.default_rng(seed)
    indices = np.arange(n_examples)
    rng.shuffle(indices)
    return np.array_split(indices, k)


def run_k_fold_cv(examples, k=3, seed=42):
    fold_indices = make_k_folds_indices(len(examples), k=k, seed=seed)
    fold_results = []

    for i in range(k):
        print(f"\nCross-validation fold {i+1}/{k}")

        val_idx = fold_indices[i]
        train_idx = np.concatenate([fold_indices[j] for j in range(k) if j != i])

        train_examples = [examples[idx] for idx in train_idx]
        val_examples = [examples[idx] for idx in val_idx]

        train_dataset_cv = InMemoryDataset(train_examples)
        val_dataset_cv = InMemoryDataset(val_examples)

        set_seed(seed + i)

        model = build_model(
            hidden_layers=CV_CONFIG["hidden_layers"],
            activation=CV_CONFIG["activation"]
        )

        trainer = CVTrainer(
            model=model,
            batch_size=min(CV_CONFIG["batch_size"], len(train_examples)),
            learning_rate=CV_CONFIG["learning_rate"],
            epoch=CV_CONFIG["epoch"],
            l2=CV_CONFIG["l2"]
        )

        train_losses, val_losses = trainer.train(
            train_dataset_cv,
            val_dataset_cv,
            early_stop=True,
            draw_curve=False
        )

        val_mse = trainer.evaluate(val_dataset_cv, draw_plot=False)
        val_mae = trainer.evaluate_mae_kcalmol(val_dataset_cv)

        fold_results.append({
            "fold": i + 1,
            "val_mse": val_mse,
            "val_mae": val_mae,
            "train_losses": train_losses,
            "val_losses": val_losses
        })
```

```
        print(f"Fold {i+1} validation MAE: {val_mae:.4f} kcal/mol")

    return fold_results
```

```
Creating development subset for cross-validation...
Number of materialized examples for CV: 204800
```

Cross-Validation Summary

In [16]:
```
k = 3
cv_results = run_k_fold_cv(cv_examples, k=k, seed=42)

print("\nImproved K-fold cross-validation summary:")
for res in cv_results:
    print(
        f"Fold {res['fold']}: "
        f"Val MSE = {res['val_mse']:.6f}, "
        f"Val MAE = {res['val_mae']:.4f} kcal/mol"
    )

cv_maes = [res["val_mae"] for res in cv_results]
print(f"\nCross-validation MAE mean ± std = {np.mean(cv_maes):.4f} ± {np.std(cv_mae
```

```
Cross-validation fold 1/3
Sequential - Number of parameters: 230404
Initialize CV training data...
100%|██████████| 20/20 [00:33<00:00,  1.69s/it]
Fold 1 validation MAE: 1.9212 kcal/mol

Cross-validation fold 2/3
Sequential - Number of parameters: 230404
Initialize CV training data...
100%|██████████| 20/20 [00:34<00:00,  1.72s/it]
Fold 2 validation MAE: 1.8958 kcal/mol

Cross-validation fold 3/3
Sequential - Number of parameters: 230404
Initialize CV training data...
100%|██████████| 20/20 [00:33<00:00,  1.70s/it]
Fold 3 validation MAE: 1.9472 kcal/mol

Improved K-fold cross-validation summary:
Fold 1: Val MSE = 0.000023, Val MAE = 1.9212 kcal/mol
Fold 2: Val MSE = 0.000020, Val MAE = 1.8958 kcal/mol
Fold 3: Val MSE = 0.000023, Val MAE = 1.9472 kcal/mol

Cross-validation MAE mean ± std = 1.9214 ± 0.0210 kcal/mol
```

The best model from Checkpoint 3, using hidden layers [128, 64], batch size 4096, learning rate 1e-3, epoch 10, and L2 = 1e-5, was evaluated in production mode. Across three independent runs, the model achieved a test MAE of 1.6389 ± 0.0118 kcal/mol, indicating stable performance across random initializations. A 3-fold cross-validation study on a development subset gave 1.9214 ± 0.0210 kcal/mol, showing that performance remains near

the project target across different folds. Based on these results, this configuration is selected as the final production-ready model.

In [ ]: